Gradient Inversion Attacks in Federated Learning

Deep Leakage from Gradients: Reconstructing Client Data from Shared Updates

Contents

1 Introduction **2**

2 Related Work **3**
2.1 Federated Learning . . . 3
2.2 Optimization-Based Gradient Inversion . . . 3
2.3 Closed-Form Gradient Inversion . . . 3
2.4 Defenses Against Gradient Inversion . . . 4
2.5 Positioning of This Project . . . 4

3 Problem Statement **5**
3.1 Federated Learning Setting . . . 5
3.2 Threat Model . . . 5
3.3 The Gradient Inversion Problem . . . 6

4 Methodology **7**
4.1 Victim Architectures . . . 7
4.2 DLG and iDLG . . . 7
4.3 Inverting Gradients . . . 8
4.4 R-GAP . . . 8
4.5 DP-SGD as a Defense . . . 9

5 Experimental Evaluation **10**
5.1 Evaluation Objectives . . . 10
5.2 Evaluation Scenarios . . . 10
5.2.1 Datasets and Victim Configurations . . . 10
5.2.2 Evaluation Metrics . . . 10
5.3 Results and Discussion . . . 11
5.3.1 RQ1 – Baseline Comparison on a Shared Architecture . . . 11
5.3.2 RQ2 – Scaling to a Deeper Architecture . . . 12
5.3.3 RQ3 – Robustness to Batch Size . . . 13
5.3.4 RQ4 – Effectiveness of the DP-SGD Defense . . . 15

6 Conclusion **18**

Chapter 1

Introduction

Federated Learning (FL) is a distributed training setup where many clients (e.g. phones, hospitals, organizations) collaboratively train one shared model without pooling their data on a central server - each client trains locally on its own private data and only sends back the resulting model update [1]. This is often presented as a privacy-preserving alternative to centralized training: since raw data never leaves the client, FL is commonly assumed to keep that data confidential by construction.[1]

This project studies whether that assumption actually holds. The gradient (or weight update) a client sends is not an opaque summary - it is the deterministic output of backpropagating a loss on the client's private batch through a model architecture the server already knows. If that function can be inverted, the private batch can be reconstructed from the gradient alone, without any further access to the client. This is the *gradient inversion* problem, first demonstrated as "Deep Leakage from Gradients" [2]: given a leaked gradient and a known model architecture, recover the input/label pair that produced it.

This project focuses on two broad attack strategies: *optimization-based* attacks, which search for an input/label pair that reproduces the leaked gradient, and *closed-form* attacks, which solve for it directly from the per-layer gradient equations, with no search involved. It implements one representative method from each - DLG [2]/iDLG [3] and Inverting Gradients [4] on the optimization side, R-GAP [5] on the closed-form side - on a common set of victim architectures, and evaluates both families against a DP-SGD-style [6] gradient perturbation defense.

The remainder of this report is organized as follows. Chapter 2 surveys the work this project builds on. Chapter 3 gives a formal, input/output statement of the FL setting, the threat model, and the gradient inversion problem. Chapter 4 describes the attack and defense methods implemented. Chapter 5 presents the experimental evaluation methodology and results, and Chapter 6 concludes with lessons learned.

[1] Code, experiment notebooks, and the LaTeX source of this report are available at `https://github.com/huynt119/Trustworthy_AI_Project`.

Chapter 2

Related Work

2.1 Federated Learning

FedAvg [1] is the foundational FL algorithm and the training protocol assumed throughout this project: clients run several steps of local SGD on their own data and the server aggregates the resulting updates by a weighted average (Equation 3.2 in Chapter 3). Its privacy argument is informal - clients never transmit raw data, only model updates - and is exactly the assumption the gradient inversion literature below sets out to test.

2.2 Optimization-Based Gradient Inversion

Deep Leakage from Gradients (DLG) [2] was the first to show that a single gradient can leak the private image and label that produced it. DLG treats reconstruction as an optimization problem: a dummy input and label are updated by gradient descent so that their gradient matches the leaked one under an L_2 distance, using second-order differentiation ("double backpropagation") to backpropagate through the gradient-matching loss itself. Joint optimization of the dummy label alongside the image makes convergence slow and occasionally unstable.

iDLG [3] removes that instability by observing that, for cross-entropy loss on a one-hot label, the sign pattern of the last layer's gradient identifies the ground-truth class in closed form - only the dummy image still needs to be optimized. This simplification makes convergence markedly more reliable.

Inverting Gradients [4] argues that most of the information in a gradient is carried by its *direction* rather than its magnitude, and replaces the L_2 distance with a cosine-similarity cost. It further adds a total-variation image prior and optimizes with a sign-based variant of Adam. These changes make the attack scale from small networks up to ImageNet-scale ResNet architectures, showing that gradient leakage is a practical threat even for deep convolutional networks trained with non-trivial batch sizes.

2.3 Closed-Form Gradient Inversion

R-GAP (Recursive Gradient Attack on Privacy) [5] reformulates gradient leakage as a linear algebra problem rather than a search problem. For a feed-forward network with bias-free layers and invertible activations, the weight-gradient equation at each layer relates a known quantity (the leaked per-layer weight gradient) to the layer's input, which can be solved for directly; recursing this from the output layer back to the input recovers the private sample with no optimization

loop. R-GAP additionally proposes a *rank analysis* that predicts, from the architecture alone, whether a given layer's input is uniquely recoverable, without running the attack at all.

2.4 Defenses Against Gradient Inversion

DP-SGD [6] was originally proposed to give differential-privacy guarantees during training, by clipping each per-example gradient to a norm C and adding Gaussian noise with multiplier σ. The same clip-and-noise procedure is widely reused, as in this project, as an ad hoc empirical defense against gradient inversion: since it is applied directly to the gradient the attacker observes, it perturbs the attacker's input rather than any property of the model.

2.5 Positioning of This Project

This project contributes an empirical comparison, under matched conditions, of optimization-based attacks (DLG/iDLG, and separately Inverting Gradients) and a closed-form attack (R-GAP) across comparable victim architectures, together with an evaluation of how all three degrade under the same DP-SGD defense budget.

Chapter 3

Problem Statement

3.1 Federated Learning Setting

FL trains a shared model across K clients, where client k keeps a private dataset $\mathcal{D}_k$ and the server optimizes the global objective

$$\min_w \; F(w) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(w), \qquad F_k(w) = \frac{1}{n_k} \sum_{i \in \mathcal{D}_k} \ell(w; x_i, y_i), \tag{3.1}$$

where w are the shared parameters and $n_k = |\mathcal{D}_k|$, $n = \sum_k n_k$. In round t, the server sends w_t to a subset of clients; each client trains locally and returns an update $\Delta w_k^{(t)}$ (or, in the single-step case this project focuses on, a gradient $g_k^{(t)} = \nabla_w F_k(w_t)$ scaled by the learning rate). The server aggregates with the FedAvg rule

$$w_{t+1} = w_t + \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} \Delta w_k^{(t)}. \tag{3.2}$$

Only $\Delta w_k^{(t)}$ (equivalently $g_k^{(t)}$) ever leaves the client - never x_i or y_i directly.

3.2 Threat Model

We consider an *honest-but-curious* attacker: it follows the FL protocol exactly as specified, but additionally tries to extract private data from whatever it legitimately observes. Two placements are relevant, and both reduce to the same observation:

- **A curious server**, which by design receives every client's update $\Delta w_k^{(t)}$ (or gradient $g_k^{(t)}$) each round.
- **An eavesdropper** on the client–server channel, which intercepts the same quantity in transit.

What the attacker knows: the model architecture and loss function $\ell(\cdot\,;\cdot)$, the current global weights w_t, and the leaked gradient (or update) itself. **What the attacker does not know:** the private batch $\{(x_i, y_i)\}$ that produced that gradient. The attacker makes no other assumption and does not deviate from the protocol - it is purely a passive observer of data every FL deployment already produces.

3.3 The Gradient Inversion Problem

Formally, for a private batch of size B,

$$g = \nabla_w \, \ell\big(w_t;\, \{x_i, y_i\}_{i=1}^{B}\big) \tag{3.3}$$

is the quantity the attacker observes (for $B = 1$ this is simply $\nabla_w \ell(w_t; x, y)$). Equation (3.3) is a deterministic function of the private batch and the known weights w_t; it is not necessarily one-way. The **gradient inversion problem** is to invert it:

> **Input:** the model architecture and loss ℓ, the weights w_t, and the leaked gradient g.
> **Output:** an estimate $\{(\hat{x}_i, \hat{y}_i)\}_{i=1}^{B}$ of the private batch.

The two attack families surveyed in Chapter 2 answer this input/output contract differently. Optimization-based attacks pose it as a search problem,

$$\min_{\hat{x},\hat{y}} \; D\big(g', g\big), \qquad g' = \nabla_w \, \ell(w_t; \hat{x}, \hat{y}), \tag{3.4}$$

for some distance or similarity measure $D(\cdot, \cdot)$ between gradient vectors, solved by gradient descent on $(\hat{x}, \hat{y})$. Closed-form attacks instead treat Equation (3.3) as a system of per-layer equations and solve directly for $\{\hat{x}_i\}$ by linear algebra, with no search and no loss D to minimize. Chapter 4 details both routes for each method implemented in this project. How the quality of $(\hat{x}, \hat{y})$ is measured against the true (x, y) - and under what gradient-perturbation defense budget - is defined separately in Chapter 5, since it is a property of the evaluation, not of the problem statement itself.

Chapter 4

Methodology

4.1 Victim Architectures

Three victim architectures are used across the attacks studied in this project:

- **LeNet-style CNN** - four convolutional layers followed by a fully-connected head. Two variants are used: the original DLG formulation (bias, sigmoid activations), used for DLG/iDLG and, under the same architecture, initial weights, and image, for Inverting Gradients, providing a common architecture for the three-attack comparison later in this chapter; and a bias-free, LeakyReLU variant required by R-GAP (referred to here as LeNetRGAP), since R-GAP's per-layer equations assume an affine map with no additive bias and an invertible activation; its output layer is a single logit trained with binary logistic loss, since R-GAP's closed-form output-layer inversion is only defined for a scalar, sigmoid-activated loss.
- **ResNet-18** - used for Inverting Gradients, to test attack scalability on a deeper, residual architecture closer to what is deployed in practice.
- **FCN3** - a three-layer, bias-free, fully-connected network (1000, 100, 1 units, LeakyReLU throughout), used alongside LeNetRGAP for R-GAP as a point of comparison: it has no convolutional layers, and therefore no extra forward-pass constraints beyond the per-layer gradient equations themselves.

4.2 DLG and iDLG

DLG [2] initializes a dummy sample $(\hat{x}, \hat{y})$ from random noise and optimizes

$$\min_{\hat{x},\hat{y}} \ \left\|g' - g\right\|_2^2, \qquad g' = \nabla_w \, \ell(w_t; \hat{x}, \hat{y}), \tag{4.1}$$

by gradient descent on $(\hat{x}, \hat{y})$ jointly. Because g' is itself a gradient, minimizing Equation (4.1) requires differentiating through $\nabla_w \ell$ a second time ("double backpropagation"); both $\hat{x}$ and $\hat{y}$ are free, continuous optimization variables, including the label, which DLG only relaxes to a one-hot vector after optimization.

iDLG [3] removes the label from the optimization. For cross-entropy loss on a one-hot label y, the gradient of the loss with respect to the pre-softmax logit z_c of class c is $\partial \ell / \partial z_c = p_c - \mathbb{1}[c = y]$, where p_c is the predicted probability. Since $p_c \in (0, 1)$, this quantity is negative only for the true class, so

$$\hat{y} = \arg\min_c \ \left(\frac{\partial \ell}{\partial z_c} \right), \tag{4.2}$$

recovers the ground-truth label in closed form from the sign pattern of the last layer's gradient alone. Only $\hat{x}$ is then optimized against Equation (4.1), which converges markedly faster and more reliably than DLG's joint optimization.

4.3 Inverting Gradients

Inverting Gradients [4] replaces the L_2 distance with a cosine-similarity cost and adds a total-variation image prior:

$$\min_{\hat{x}} \left(1 - \frac{\langle g', g \rangle}{\|g'\| \, \|g\|}\right) + \alpha \, \mathrm{TV}(\hat{x}), \qquad g' = \nabla_w \, \ell(w_t; \hat{x}, y), \tag{4.3}$$

where $\mathrm{TV}(\hat{x}) = \sum_{i,j} |\hat{x}_{i+1,j} - \hat{x}_{i,j}| + |\hat{x}_{i,j+1} - \hat{x}_{i,j}|$ is the total variation of the image - the sum of absolute differences between horizontally and vertically neighboring pixels - which penalizes non-smooth reconstructions. The label y is recovered the same way as in iDLG (Equation (4.2)), and $\hat{x}$ is constrained to the valid image range $[0, 1]^d$ by a box constraint. Two choices make Equation (4.3) scale to deep networks where DLG's L_2 cost does not: the cosine-similarity term depends only on the *direction* of g', not its magnitude, which is otherwise dominated by whichever layer happens to have the largest gradient norm; and the optimizer applies Adam updates using only the *sign* of the computed gradient, which prevents any single coordinate's magnitude from controlling the step size. This project evaluates Equation (4.3) on an untrained ResNet-18 victim model, and separately on the shared LeNet architecture described above, under the same weights and image used for DLG/iDLG; the latter configuration is the one used in the defense comparison later in this chapter.

4.4 R-GAP

R-GAP [5] targets a feed-forward network of L bias-free layers with invertible elementwise activation σ: writing $a_0 = x$ for the input and, for $l = 1, \ldots, L$,

$$z_l = W_l \, a_{l-1}, \qquad a_l = \sigma(z_l), \tag{4.4}$$

the weight gradient at layer l for a single sample ($B = 1$) is the outer product

$$\frac{\partial \ell}{\partial W_l} = \frac{\partial \ell}{\partial z_l} \, a_{l-1}^{\top}. \tag{4.5}$$

The leaked gradient $\partial \ell / \partial W_l$ is known for every layer; $\partial \ell / \partial z_l$ is obtained recursively via the chain rule, $\partial \ell / \partial z_l = \sigma'(z_l) \odot \left(W_{l+1}^{\top} \, \partial \ell / \partial z_{l+1}\right)$, starting from the output layer, where $\partial \ell / \partial z_L$ is a scalar equation solved numerically (the loss-to-logit derivative is a monotonic function of the scalar logit for the binary logistic loss used here). With $\partial \ell / \partial z_l$ known, Equation (4.5) becomes a linear system for the unknown a_{l-1}, solved by least squares; recursing this from $l = L$ down to $l = 1$ recovers $a_0 = x$ - the private input itself - with *no* optimization loop: each layer's equation is solved exactly, subject to its own rank/invertibility condition. Convolutional layers are handled by first re-expressing the weight-sharing convolution as an equivalent dense (circulant) matrix multiplication, so that Equation (4.5) applies unchanged.

This project implements R-GAP on both victim architectures described above (LeNetRGAP and FCN3).

4.5 DP-SGD as a Defense

DP-SGD [6] is applied here, after the fact, as a gradient-perturbation defense: before the leaked gradient g reaches the attacker, it is clipped to a global L_2 norm C and perturbed with Gaussian noise of multiplier σ,

$$\tilde{g} = \text{clip}(g, C) + \mathcal{N}\big(0,\ (\sigma C)^2 I\big), \qquad \text{clip}(g, C) = g \cdot \min\left(1, \frac{C}{\|g\|_2}\right). \tag{4.6}$$

The same perturbed gradient $\tilde{g}$ is substituted wherever g appears in Equations (4.1), (4.3), and the R-GAP recursion above, allowing the three attacks to be compared under an identical defense budget (C, σ). Inverting Gradients is evaluated under both victim configurations introduced above: the ResNet-18 setup, and the LeNet architecture shared with DLG/iDLG. This project sweeps (C, σ) for each attack and configuration; the resulting reconstruction quality is reported in Section 5.3.4.

Chapter 5

Experimental Evaluation

5.1 Evaluation Objectives

The methods described in Chapter 4 are evaluated to answer four questions, each changing exactly one factor relative to a shared anchor configuration (LeNet, CIFAR-10, batch size 1, no defense):

1. **RQ1.** Under a matched architecture, dataset, and batch size, how do an optimization-based attack (DLG/iDLG) and a closed-form attack (R-GAP) compare in reconstruction quality and runtime?
2. **RQ2.** Does an optimization-based attack designed for scale (Inverting Gradients) still succeed when the victim architecture is deeper and closer to what is deployed in practice?
3. **RQ3.** How does each attack's reconstruction quality change when the leaked gradient is averaged over a batch of two private images instead of one?
4. **RQ4.** Can a standard gradient-perturbation defense (DP-SGD) suppress each attack, and does the noise level required to do so differ between attack families?

5.2 Evaluation Scenarios

5.2.1 Datasets and Victim Configurations

All attacks target a victim model at a single, untrained (or early-training) set of weights, consistent with the threat model of Section 3.2: the attacker observes a gradient from one of the first FL rounds, before the model has learned anything client-specific. Table 5.1 summarizes every configuration evaluated.

5.2.2 Evaluation Metrics

Reconstruction quality is reported with two standard image-similarity metrics, computed in pixel space ($[0, 1]$ range) between the reconstructed image $\hat{x}$ and the ground-truth image x:

$$\text{MSE} = \frac{1}{d}\|\hat{x} - x\|_2^2, \qquad \text{PSNR} = 10 \log_{10}\left(\frac{1}{\text{MSE}}\right) \text{ (dB)}. \tag{5.1}$$

Method	Architecture	Dataset	Batch sizes	DP-SGD swept
DLG / iDLG	LeNet (bias, sigmoid)	CIFAR-10	1, 2	Yes
Inverting Gradients	ResNet-18 (untrained)	ImageNet (sample images)	1, 2	Yes
Inverting Gradients	LeNet (shared with DLG/iDLG)	CIFAR-10	1	Yes
R-GAP	LeNetRGAP (bias-free, LeakyReLU)	CIFAR-10	1, 2	Yes (batch 1 only)
R-GAP	FCN3 (bias-free, fully-connected)	CIFAR-10 (resized)	2	No

Table 5.1: Victim configurations evaluated. The LeNet row shared by DLG/iDLG and Inverting Gradients uses identical architecture, initial weights, and image (Section 4.1), giving RQ1 and RQ4 a matched-architecture comparison point across the two attack families.

Higher PSNR (equivalently, lower MSE) indicates a reconstruction closer to the original image; a PSNR near or below 0 dB indicates the reconstruction is no more informative than noise. Two secondary metrics are used where applicable: **label accuracy** (whether the attacker's recovered label matches the ground-truth label, relevant to DLG/iDLG and Inverting Gradients, which recover a label as part of the attack) and **gradient cosine similarity** between the defended and original gradient (used in the DP-SGD sweeps below as a direct measure of how much signal the defense removes, independent of whether the attack converges).

5.3 Results and Discussion

5.3.1 RQ1 – Baseline Comparison on a Shared Architecture

DLG (Equation (4.1)), iDLG, Inverting Gradients (Equation (4.3)), and R-GAP (Equation (4.5)) are each run once on the shared LeNet configuration of Table 5.1 (the LeNetRGAP variant for R-GAP), batch size 1, no defense: the same dataset, the same image, and the same convolutional topology for all four attacks, with one exception — the bias/activation difference that R-GAP's closed-form derivation requires (Section 4.1). The comparison reports reconstruction quality (MSE, PSNR) for each method.

Method	MSE	PSNR
DLG	0.00140	28.55 dB
iDLG	0.00160	27.95 dB
Inverting Gradients	0.0022	26.48 dB
R-GAP	$< 10^{-7}$	∞ (exact)

Table 5.2: RQ1 baseline comparison: same LeNet, same CIFAR-10 image, batch size 1, no defense.

R-GAP's closed-form solve reproduces the image to within floating-point rounding error, confirmed visually in Figure 5.1: its reconstruction is indistinguishable from the ground truth. DLG and iDLG converge to a close but visibly noisy approximation after 300 iterations of optimization, and Inverting Gradients – tuned for scale rather than precision on small networks – trails both on this LeNet configuration. R-GAP is also the fastest attack by a wide margin, since it has no per-pixel optimization loop (Section 4.4); its 57.3 s is spent almost entirely on the one scalar optimization at the output layer, not on the four closed-form linear solves.

Figure 5.2 tracks how DLG and iDLG reach that final reconstruction: both start from the same random noise and converge toward the ground truth over the 300-iteration L-BFGS run,

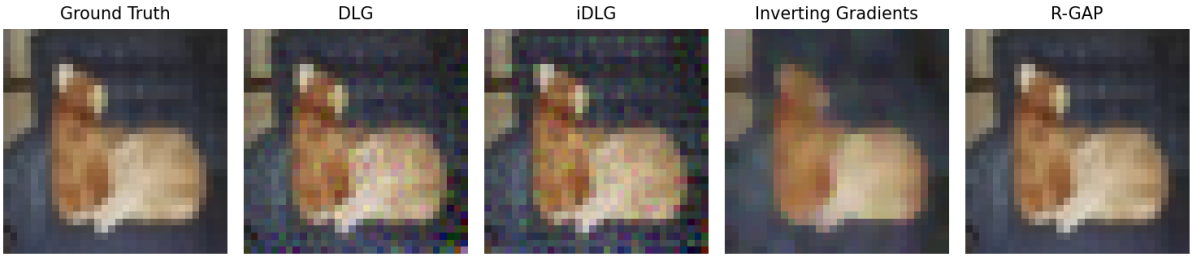


Figure 5.1: Ground truth and reconstructions from all four attacks on the shared LeNet configuration. R-GAP's reconstruction is pixel-exact to floating-point precision.

with iDLG's single-variable optimization visibly settling into the right shape a few milestones earlier than DLG's joint image-and-label search.

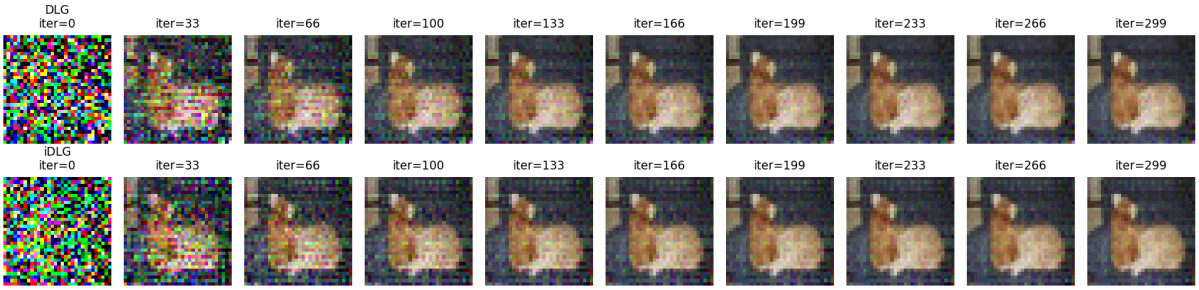


Figure 5.2: DLG (top) and iDLG (bottom) reconstructions at 10 evenly-spaced iterations of the same optimization run shown in Figure 5.1.

Numerical precision. R-GAP's exactness is conditional on computing the entire attack in float64. Repeating the same attack in float32 (PyTorch's default) on identical weights and gradient yields MSE $= 91.25$, PSNR $= -19.60$ dB – pure noise. Figure 5.3 localizes the failure: the float32 and float64 reconstructions agree closely through all four intermediate layers, and diverge only at the final step (recovering the image itself from the first convolutional layer's input), where the smallest singular value of that layer's linear system is small enough for float32 rounding error to be amplified into dominant noise. DLG and iDLG have no equivalent failure mode at this precision, since gradient descent is not solving an exact linear system at any step.

5.3.2 RQ2 – Scaling to a Deeper Architecture

Inverting Gradients (Equation (4.3)) is run on an untrained ResNet-18, 11.7M-parameter victim model and a 224×224 image, a setting an order of magnitude larger than the LeNet configuration of RQ1. Section 4.3 already explains, at the level of the optimization objective, why the cosine cost and sign-based gradient update were introduced specifically to make this scale tractable; this section reports whether that design succeeds in practice, by the same MSE/PSNR metrics as RQ1.

With 4 random restarts of 24,000 iterations each, the attack reaches MSE $= 0.2944$, PSNR $= 17.33$ dB. Figure 5.4 shows the reconstruction is visibly identifiable (pose, fur color, collar pattern) despite the much larger parameter count and image resolution than RQ1's LeNet setting, confirming that the cosine cost and sign-based gradient update are sufficient to keep the attack

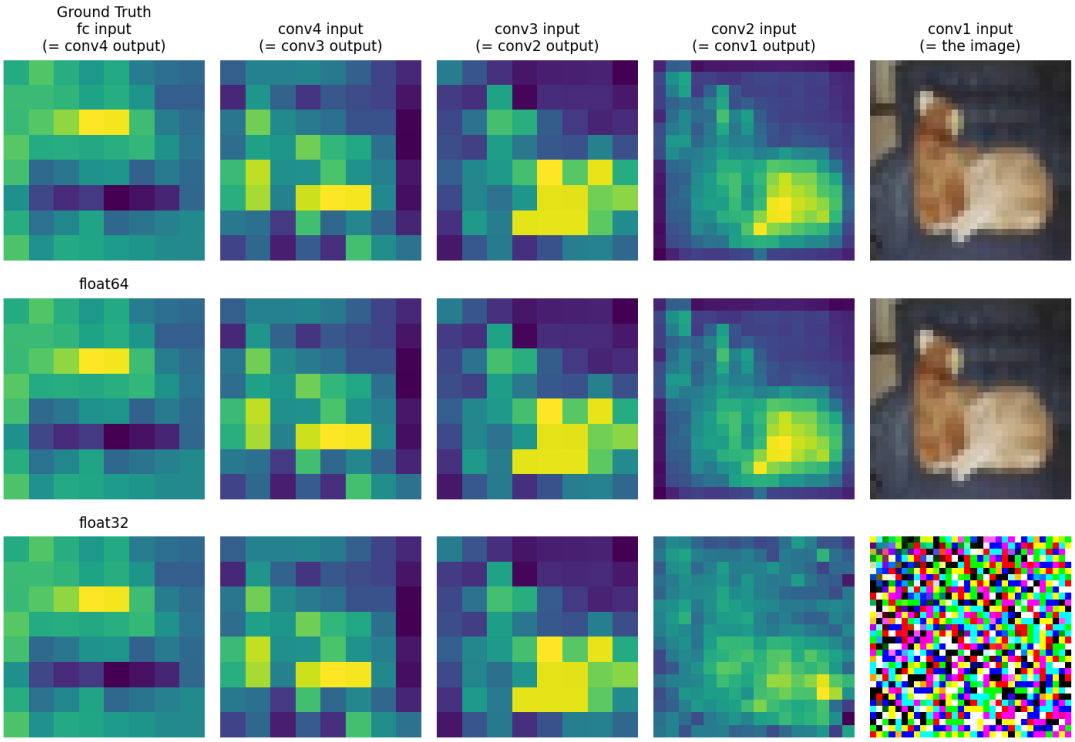


Figure 5.3: R-GAP's intermediate reconstructions (deepest layer to final image, left to right) in float64 vs. float32. The two agree until the last step.

tractable at this scale – though at markedly lower fidelity than RQ1's Inverting Gradients result on the smaller LeNet (PSNR 26.48 dB; Section 5.3.1).



Figure 5.4: Inverting Gradients on an untrained ResNet-18 (ImageNet image, batch size 1, no defense).

5.3.3 RQ3 – Robustness to Batch Size

Each attack is compared against its own batch-size-1 result when the leaked gradient instead averages two private images: DLG/iDLG, Inverting Gradients (on both of its victim configurations), and R-GAP. R-GAP is evaluated on both victim architectures from Table 5.1, since Section 4.4 already identifies a specific, architecture-dependent reason batch size might affect it differently from the optimization-based attacks: its recursive equations assume a single sample,

and at batch size > 1 a convolutional architecture additionally imposes forward-pass constraints that a purely fully-connected one (FCN3) does not. This RQ tests whether that predicted asymmetry (LeNetRGAP vs. FCN3) is actually observed, alongside the batch-size degradation of the two optimization-based attacks.

Method (architecture)	Batch 1 (PSNR)	Batch 2 (mean PSNR)
DLG (LeNet)	28.55 dB	16.29 dB
iDLG (LeNet)	27.95 dB	16.65 dB
Inverting Gradients (ResNet-18)	17.33 dB	16.35 dB

Table 5.3: Reconstruction quality at batch size 1 vs. 2, same architecture and attack.

Both optimization-based attacks lose roughly 11–12 dB going from batch 1 to batch 2 on LeNet, while Inverting Gradients on ResNet-18 loses under 1 dB – its batch-1 result is already far from perfect at this scale (RQ2), leaving less margin left to lose. Figure 5.5 shows both ResNet-18 batch-2 reconstructions remain individually identifiable.

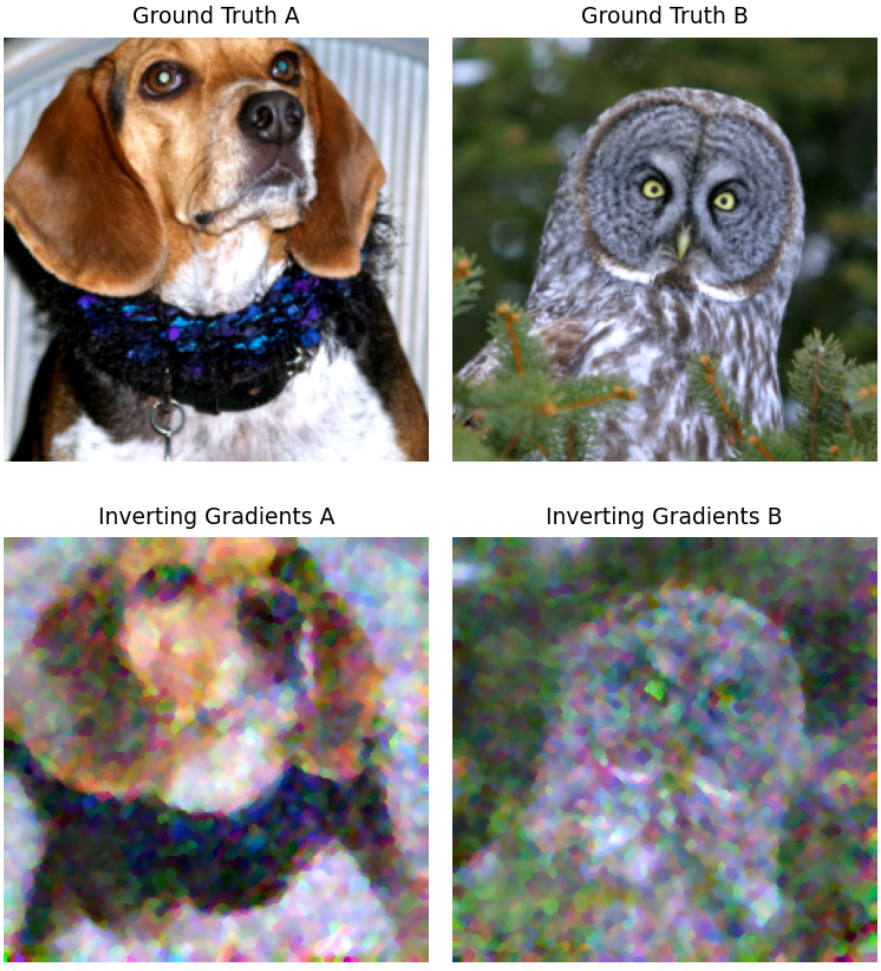


Figure 5.5: Inverting Gradients, ResNet-18, batch size 2.

R-GAP at batch size 2: the linear-combination property. With $B = 2$, the leaked weight gradient is an average over both images, and Equation (4.5) no longer uniquely determines either

input on its own. For a purely linear (bias-free, fully-connected) network, the recursive solution R-GAP recovers is exactly the *average* of the two inputs, $\frac{1}{2}(x_A + x_B)$ – a direct algebraic consequence of Equation (4.5) being linear in the input. A convolutional architecture adds the extra forward-pass constraints of Equation (4.4) at every layer, which the blended, two-image gradient signal does not consistently satisfy. Table 5.4 confirms this: on FCN3, R-GAP's final-stage output matches the plain pixel average almost exactly; on LeNetRGAP (CNN), it matches neither individual image nor their average.

Architecture	$\lvert\cos\rvert$ vs. image A	$\lvert\cos\rvert$ vs. image B	$\lvert\cos\rvert$ vs. average $(A{+}B)/2$
LeNetRGAP (CNN)	0.062	0.055	0.061
FCN3 (linear only)	0.959	0.967	1.000

Table 5.4: R-GAP batch-2 final-stage reconstruction, cosine similarity against ground truth.



Figure 5.6: R-GAP batch-2 reconstruction: LeNetRGAP/CNN (top, matches neither image nor their average) vs. FCN3 (bottom, recovers almost exactly the pixel average $(A{+}B)/2$).

At batch size 1, R-GAP reconstructs this same architecture's input exactly (Section 5.3.1); at batch size 2 the CNN variant's reconstruction is uncorrelated with either image or their average ($\lvert\cos\rvert \approx 0.06$ in Table 5.4), a sharper collapse than either optimization-based attack shows going from batch 1 to batch 2.

5.3.4 RQ4 – Effectiveness of the DP-SGD Defense

The DP-SGD defense of Equation (4.6) is applied to the leaked gradient before each attack ever observes it, sweeping clipping thresholds $C = 0.5\|g\|$ and $C = 1.0\|g\|$ (half the gradient's own norm, and the exact boundary at which clipping has no effect) together with a grid of noise multipliers σ. Thresholds are set relative to each configuration's own gradient norm so that clipping strength is comparable across attacks whose gradients live at very different absolute scales. Inverting Gradients is swept on both the shared LeNet (for a direct, matched-architecture comparison against DLG/iDLG and R-GAP) and ResNet-18 (to check whether the defense still

holds at scale); R-GAP uses a wider range of noise multipliers (Section 4.5), since its sensitivity to numerical perturbation would otherwise place every sampled σ past its breaking point. Each configuration here also runs fewer optimization iterations than the undefended baselines of RQ1/RQ2, so the $\sigma = 0$ row below is a lower-budget reference point, not directly comparable to those undefended numbers.

At the LeNet configuration shared by DLG/iDLG and Inverting Gradients, clipping alone already separates the two cost functions as Section 4.3 predicts: iDLG's L_2 cost is sensitive to the gradient's absolute magnitude, so $C = 0.5\|g\|$ collapses it even at $\sigma = 0$ (PSNR -105 dB), while Inverting Gradients' scale-invariant cosine cost gives nearly identical PSNR at both clipping settings (Table 5.5). Increasing σ then degrades both attacks, but at different rates: iDLG collapses sharply between $\sigma = 10^{-4}$ and $\sigma = 10^{-3}$, while Inverting Gradients on the same architecture stays above 17 dB at $\sigma = 10^{-3}$ – roughly an order of magnitude more noise tolerance. On ResNet-18, Inverting Gradients' defended PSNR stays in a narrow 9–13 dB band across the full grid, close to its $\sigma = 0$ value of 12.72 dB – the defense adds little further degradation beyond what the reduced iteration count already costs.

	PSNR (dB) at $C = 1.0\|g\|$ (no clipping)			
Method (architecture)	$\sigma = 0$	10^{-4}	10^{-3}	10^{-2}
iDLG (LeNet)	16.14	14.05	-8.11	-18.83
Inverting Gradients (LeNet)	27.94	27.32	17.34	10.75
Inverting Gradients (ResNet-18)	12.72	12.17	9.71	9.43

Table 5.5: DP-SGD sweep, no-clip row ($C = 1.0\|g\|$). The $C = 0.5\|g\|$ row is omitted: it is nearly identical to this row for Inverting Gradients (cosine cost) but collapses iDLG entirely (L_2 cost), as noted above.

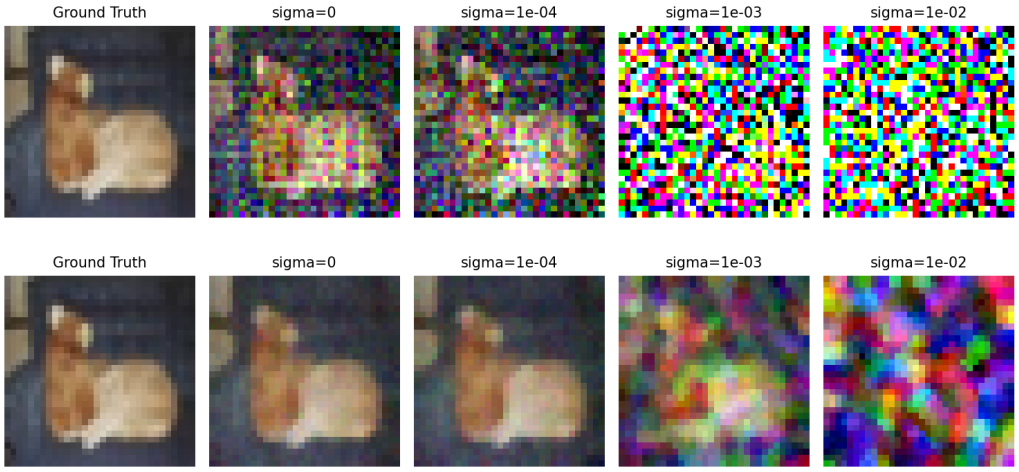


Figure 5.7: Reconstruction vs. DP-SGD noise, same LeNet architecture: iDLG (top) vs. Inverting Gradients (bottom).

R-GAP under DP-SGD. Table 5.6 reports R-GAP's own, finer-grained sweep (LeNetRGAP, same image as RQ1). The attack is exact at $\sigma = 0$ and still reaches 31.0 dB at $\sigma = 10^{-7}$, but collapses to -19.5 dB at $\sigma = 10^{-6}$ – just one order of magnitude more noise. This breaking point sits three to four orders of magnitude below iDLG's ($\sigma \approx 10^{-3}$–10^{-4}) on a comparable LeNet

architecture: the noise level needed to defeat R-GAP leaves DLG/iDLG's reconstruction visibly unaffected. This matches Section 4.4's mechanism – an exact recursive solve cannot average out perturbation the way gradient descent does across hundreds of steps – giving DP-SGD a use it otherwise lacks: a noise level too small to touch the optimization-based attacks already defeats R-GAP outright.

	PSNR (dB), $C = 1.0\|g\|$ (no clipping)					
σ	0	10^{-9}	10^{-8}	10^{-7}	10^{-6}	10^{-5}
R-GAP (LeNetRGAP)	∞	73.2	51.6	31.0	-19.5	-18.5

Table 5.6: R-GAP DP-SGD sweep (first six of eight σ values; 10^{-4} and 10^{-3} remain collapsed at -27.2 and -42.4 dB).

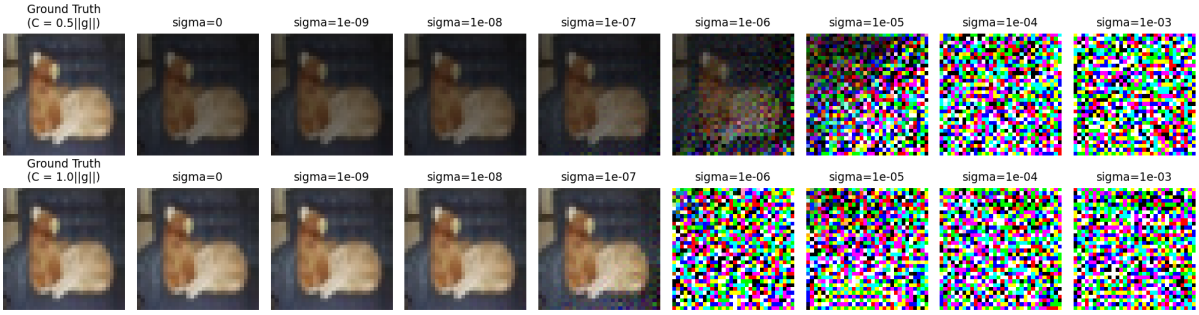


Figure 5.8: R-GAP reconstruction vs. DP-SGD noise, both clipping settings. The break between $\sigma = 10^{-7}$ and $\sigma = 10^{-6}$ is visible by eye.

Chapter 6

Conclusion

This project implemented and compared two families of gradient inversion attacks against Federated Learning's informal privacy claim — optimization-based (DLG, iDLG, Inverting Gradients) and closed-form (R-GAP) — under four controlled comparisons spanning architecture, scale, batch size, and a DP-SGD defense (Chapter 5).

> *[Pending final data — a concise summary of the main quantitative conclusions across RQ1–RQ4 will be written here once the results are reviewed.]*

Two extensions follow directly from this comparison. Pairing the attack-side measurements of Section 4.5 with a training-side utility measurement would show whether a noise level sufficient to defeat these attacks is one a real deployment could afford. Separately, R-GAP's own rank-analysis framework (Section 4.4) could be used to predict, for architectures not tested here (deeper convolutional networks, BatchNorm, ReLU), whether a closed-form attack is even theoretically possible before porting the attack itself.

Bibliography

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. Agüera y Arcas. *Communication-Efficient Learning of Deep Networks from Decentralized Data*. Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS 2017), PMLR 54, pp. 1273–1282.

[2] L. Zhu, Z. Liu, S. Han. *Deep Leakage from Gradients*. Advances in Neural Information Processing Systems 32 (NeurIPS 2019), pp. 14747–14756. arXiv:1906.08935.

[3] B. Zhao, K. R. Mopuri, H. Bilen. *iDLG: Improved Deep Leakage from Gradients*. arXiv:2001.02610, 2020.

[4] J. Geiping, H. Bauermeister, H. Dröge, M. Moeller. *Inverting Gradients — How Easy Is It to Break Privacy in Federated Learning?* Advances in Neural Information Processing Systems 33 (NeurIPS 2020).

[5] J. Zhu, M. B. Blaschko. *R-GAP: Recursive Gradient Attack on Privacy*. International Conference on Learning Representations (ICLR 2021). arXiv:2010.07733.

[6] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, L. Zhang. *Deep Learning with Differential Privacy*. Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS 2016), pp. 308–318. arXiv:1607.00133.